

# CASCADE-V — Technical Approach

Multi-stage audio attribution with auditable per-receipt invariants over an embedding-proxy value function

Gaurav Gupta

April 29, 2026

## 1. Problem statement

Given an AI-generated audio output  $t$  and a catalog of source stems  $\{s_1, \dots, s_N\}$ , attribute  $t$  to a subset of contributing stems with non-negative weights  $w \in \mathbb{R}_{\geq 0}^N$ ,  $\sum_i w_i = 1$ . Emit, per receipt, machine-checkable invariants over  $w$  that an auditor can verify independently. The attribution runs at a target wall-clock of  $\sim 100$  ms on Apple Silicon at  $N = 5\,000$ .

What this system **provides**:

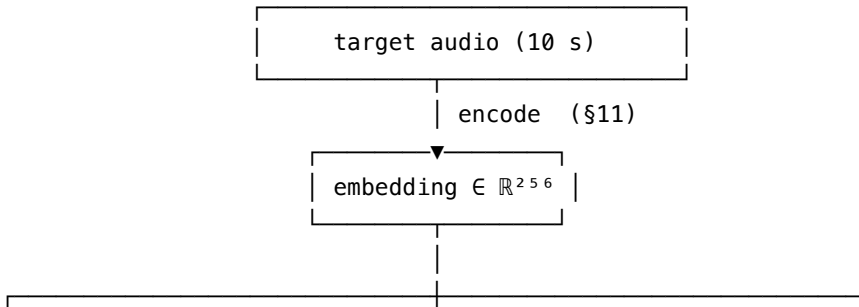
- A per-receipt SMT-LIB certificate (sha256-hashed) that the output  $w$  satisfies three classical Shapley axioms over a *value-function proxy*  $v$  defined in §7.
- Topologically-ordered pre/post-condition validators between pipeline stages. (US 12,536,365 B1 — Cascade Validation Protocol.)
- An optional policy layer (currencies, §10) with its own dignity invariants on the *payout* allocation, separately verifiable.

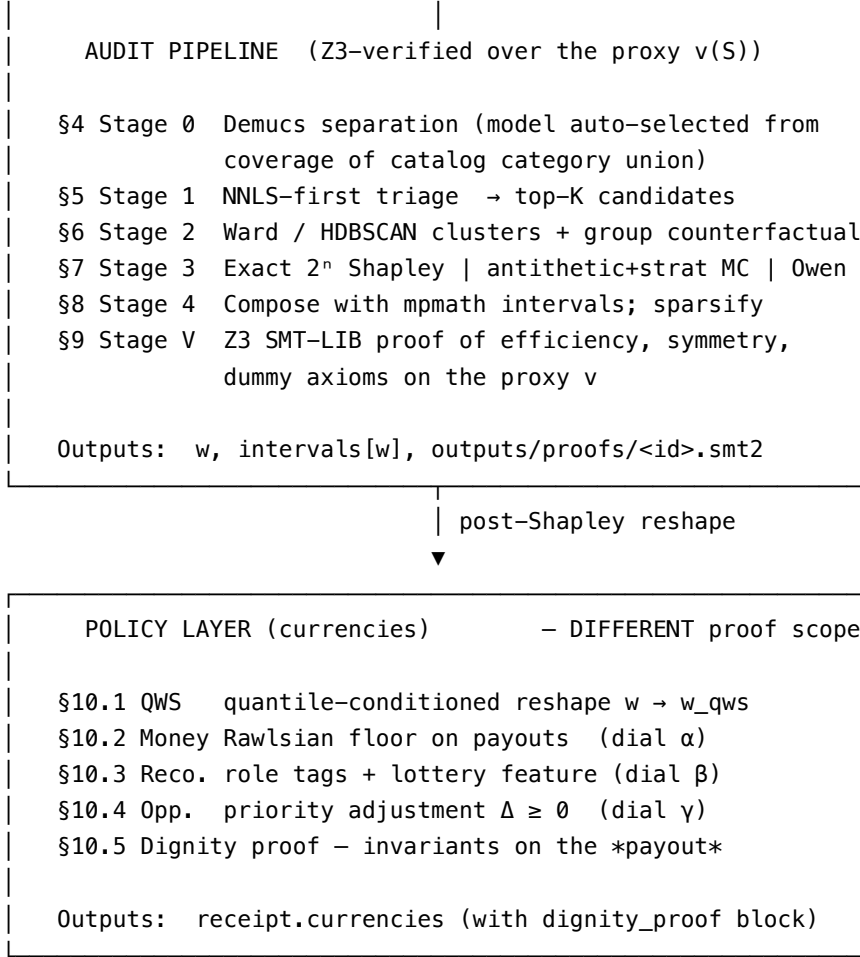
What this system **does not provide**:

- A proof that  $w$  matches true creator contribution. The Z3 certificate is a check on the output vector, not on attribution correctness — see §13(a).

## 2. System overview

The pipeline is two layers with **disjoint proof scopes**: an *audit pipeline* whose output is a Shapley allocation certified by Z3, and a *policy layer* whose output is the actual payout, certified by a separate dignity proof. Conflating the two is the cardinal sin (§13(b), §13(d)).





### 3. Notation and data model

| Symbol                                              | Meaning                                               |
|-----------------------------------------------------|-------------------------------------------------------|
| $t \in \mathbb{R}^D$                                | target embedding, L2-normalised, $D = 256$            |
| $E = (e_1, \dots, e_N) \in \mathbb{R}^{N \times D}$ | catalog embeddings, L2-normalised rows                |
| $v : 2^N \rightarrow [0, 1]$                        | value function (proxy; defined in §7)                 |
| $\phi_i$                                            | Shapley value of source $i$ over $v$                  |
| $\rho(C) = v(N) - v(N \setminus C)$                 | cluster $C$ counterfactual                            |
| $W_C$                                               | normalised cluster weight                             |
| $w \in \mathbb{R}_{\geq 0}^N, \sum w_i = 1$         | post-sparsification per-source weights                |
| $[\underline{w}_i, \overline{w}_i]$                 | mpmath interval bounds on $w_i$                       |
| $\alpha, \beta, \gamma \in [0, 1]$                  | currency dials (fairness / recognition / opportunity) |

A `creator_id` is a derived attribute: when the catalog comes from Slakh2100 the implementation derives it from `(inst_class, midi_program // 8)` — i.e. **rendering-chain identity**, not human producer identity. Two distinct producers using the same soundfont share a `creator_id`; that fact is load-bearing in §11 (supervised contrastive loss) and §13(e) (limitation).

Files of record for the data: `data/catalog/`, `data/catalog_metadata.json`, `data/ground_truth.json`, `data/catalog_embeddings.npy`. Receipts: `outputs/receipts/<id>.json`. Proofs: `outputs/proofs/<id>.smt2`. Eval results: `outputs/results.csv`. Training log: `logs/training.json`.

## 4. Stage 0 — Source separation

### Inputs / outputs

Input: raw mix audio (any sample rate; auto-resampled to the model’s expected SR). Output: dictionary `stems[name] → np.ndarray` of length  $N_{\text{stems}}$  where  $N_{\text{stems}}$  depends on the chosen Demucs model (4 or 6).

### Algorithm

```

audio → resample to model_sr          (default 44.1 kHz, stereo)
      ▼
      Hybrid Demucs / Demucs-6s       (Meta, 2019–2023)
      ▼
      per-stem mono audio at source SR

```

Aggregation across stems happens at the end of the pipeline (§8). Per-stem RMS energy  $E_k = \sqrt{\frac{1}{T} \sum_{\tau} s_k(\tau)^2}$  gates which stems get attributed: stems with  $E_k < 5 \cdot 10^{-3}$  are skipped. The aggregation share is  $\eta_k = E_k / \sum_{j \text{ not skipped}} E_j$ .

### Rationale

- Stage 1 cosine retrieval finds the dominant source in a mix and misses masked secondary contributors. Splitting first means each per-stem attribution sees a cleaner signal.
- Model auto-selection: with 12 catalog categories (kick, snare, hat, bass, sub\_bass, lead, pluck, arp, pad, ambient, vocal\_chop, fx), the 4-stem `htdemucs` collapses 6 categories into “other”; `htdemucs_6s` (drums / bass / piano / guitar / vocals / other) covers 11/12. The selector (`src/cascade_v/stages/demucs_selector.py:select_demucs_model`) reads the catalog category union and picks the model with the highest non-“other” coverage; ties broken by smaller model.

### Configuration knobs

| Setting                            | Default | Acceptable range                                                                                           |
|------------------------------------|---------|------------------------------------------------------------------------------------------------------------|
| <code>USE_DEMUCS_SEPARATION</code> | True    | bool                                                                                                       |
| <code>DEMUCS_MODEL</code>          | "auto"  | auto, <code>htdemucs</code> , <code>htdemucs_6s</code> , <code>htdemucs_ft</code> , <code>mdx_extra</code> |

### Known limits

- Stage-0 quality directly bounds the rest of the pipeline. Demucs was trained on real music; on the synthetic catalog (oscillator stems) most energy ends up in the “other” stem regardless of model choice — see §13(d).
- `htdemucs_6s` adds ~3 s per attribution vs the 4-stem default.

## 5. Stage 1 — Triage

### Inputs / outputs

Input: target embedding  $t$ , catalog embeddings  $E$ , optional (catalog\_bpm, catalog\_keys). Output: top- $K$  candidate indices ordered by ranking score; default  $K = 40$ .

### Algorithm

1. (optional) BPM/key gate:
  - drop catalog row  $i$  if  $|bpm_i - bpm_t| > \tau_{bpm}$   
or  $key_i$  not compatible with  $key_t$
2. base similarities:  $s_i = e_i \cdot t$  (length  $N$ )  
centred:  $s'_i = s_i - \beta_{cal}$   $\beta_{cal} = \text{mean}(s_J), J \subset [N]$
3. pre-NNLS probe on global top-`shortlist` (default  $M = 80$ ):  
 $\hat{w}_{probe} = \text{NNLS\_solve}(E_{shortlist}, t)$   
if  $(\hat{w}_{probe} > 1e-6).sum() \geq 5$  : skip residual passes
4. (optional) residual matching pursuit,  $n$  passes:  
 $u \leftarrow t - \sum_{\{j \in \text{top-3 of pass}\}} \hat{w}_j \cdot e_j$  (re-normalised)  
repeat scoring with  $u$  in place of  $t$ ; accumulate candidates
5. shortlist NNLS:  $\hat{w} = \arg \min_{\{w \geq 0\}} ||t - \sum_i w_i e_i||^2_2$  over shortlist  
ranking:  $r_i = (\hat{w}_i / \sum \hat{w}) + 1e-4 \cdot s'_i$   
return top- $K$  of  $r$

### Rationale (for NNLS-first ranking)

Two facts shape the choice. (i) Catalogues are  $\sim 100\times$  larger than the realistic contributor count, so the prior is **few real contributors**. NNLS is naturally sparse and matches that prior. (ii) Cosine ranks every catalog entry on a continuous scale; a not-actually-contributing stem can score 0.92 purely because it's the same instrument category as a real contributor. Using  $r_i = \hat{w}_i + 0.05 s'_i$  (the v0.1 formulation) made cosine decisive on ties (most of the catalog has  $\hat{w}_i = 0$ ), producing a long tail of false positives ranked by raw cosine. The current  $10^{-4}$  coefficient demotes cosine to a pure tie-breaker.

### Configuration knobs

| Setting                   | Default | Notes                                                           |
|---------------------------|---------|-----------------------------------------------------------------|
| TRIAGE_TOP_K              | 40      | must be $\geq$ realistic contributor count; $\leq$ catalog size |
| USE_BPM_KEY_FILTER        | False   | no-op when metadata is missing                                  |
| BPM_FILTER_TOLERANCE      | 3.0 BPM |                                                                 |
| nnls_active_skip_residual | 5       | hardcoded; see §14                                              |
| n_residual_passes         | 2       | overridden to 0 when probe finds $\geq$ skip threshold          |

## Known limits

- The threshold of 5 for skipping residual passes is unmotivated and likely overfit to the synthetic mix distribution (3–6 contributors per output). Behaviour on catalogs with different mix complexity is unmeasured. See §14.
- BPM/key gate is off by default; reported metrics in §12 are *without* it. We do not yet have evidence on the magnitude of the false-positive reduction from enabling it.

## 6. Stage 2 — Grouping and counterfactual

### Inputs / outputs

Input:  $K \times D$  candidate embeddings, target  $t$ , value-function temperature  $T$ . Output: cluster labels per candidate, per-cluster weight  $W_C$  summing to 1.0 over surviving clusters.

### Algorithm

```
labels = ward_or_hdbscan( pdist(candidates) )
v = make_value_function(target, candidates, T) # §7
v_full = v( (0, 1, ..., K-1) )
for each cluster C in unique(labels):
    ρ(C) = v_full - v( all-indices-not-in-C )
W = clip(ρ, 0, ∞) ; W = W / sum(W)
# sparsify
W = where(W >= MIN_CLUSTER_WEIGHT, W, 0) ; W = W / sum(W)
```

The cluster-removal value:

$$\rho(C) = v(N) - v(N \setminus C), \quad W_C = \frac{[\rho(C)]_+}{\sum_{C'} [\rho(C')]_+}.$$

HDBSCAN noise points (label  $-1$ ) are promoted to singleton clusters so the partition stays complete.

### Rationale

Two stems by the same producer cluster in embedding space. Pure instance-level Shapley splits credit between them — correct at the source level, wrong at the creator level. Group-level counterfactuals assign one weight to the cluster, then Stage 3 splits within.

### Configuration knobs

| Setting                    | Default |
|----------------------------|---------|
| CLUSTERING_METHOD          | "ward"  |
| CLUSTER_DISTANCE_THRESHOLD | 0.6     |
| MIN_CLUSTER_WEIGHT         | 0.07    |

## Known limits

- The Ward cut threshold 0.6 corresponds to cosine  $\approx 0.82$  boundary; on a catalog where same-creator stems sit at higher cosine (e.g. tighter producer clusters in real audio) the threshold may merge distinct creators. Sensitivity not measured.
- $\rho(C)$  is computed against the *proxy* value function (§7), not the true contribution. See §13(a).

## 7. Stage 3 — Coalitional attribution

### 7.1 Value function (the proxy)

For a coalition  $S \subseteq \{1, \dots, K\}$  over candidate embeddings:

$$v(S) = \begin{cases} 0 & \text{if } S = \emptyset \\ [\max(0, \cos(\bar{e}_S, t))]^{T/4} & \text{otherwise} \end{cases}, \quad \bar{e}_S = \frac{1}{|S|} \sum_{i \in S} e_i.$$

Bounded in  $[0, 1]$ . Negative cosine clamps to 0 (an anti-aligned coalition contributes nothing). The exponent  $T/4$  sharpens concentration on coalitions whose mean points strongly toward the target — at  $T = 8$ , the exponent is 2.

**This is a proxy.** Real audio mixing is non-linear (compression, EQ interaction, masking, phase). Mean-pooling embeddings is an *assumption* that the encoder’s representation is approximately additive over mixes. The mix-consistency loss in §11.2 trains for exactly this property on synthetic data; for real audio it holds at best approximately. Every claim downstream (Shapley over  $v$ , Z3 proof over Shapley) is therefore a claim *about the proxy*, not about creator contribution. §13(a).

The temperature  $T$  is hardcoded to 8.0 (`settings.py:SHAPLEY_VALUE_TEMPERATURE`). Sensitivity of payouts to  $T$  has not been measured (§14, §15).

### 7.2 Exact Shapley (for $|C| \leq \text{SHAPLEY\_MAX\_EXACT\_N} = 10$ )

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(n - |S| - 1)!}{n!} [v(S \cup \{i\}) - v(S)].$$

$O(2^n)$  value-function evaluations; intervals are point estimates (no uncertainty).

### 7.3 Antithetic + stratified Monte Carlo Shapley (for larger $|C|$ )

Standard permutation MC walks each  $\pi$  from empty to full, crediting each  $i$  with its marginal at the moment it joins. We add two variance reductions:

- **Antithetic:** for each random  $\pi$  also evaluate  $\pi^{-1}$ . Marginal contributions of  $(\pi, \pi^{-1})$  are negatively correlated, halving variance at equal sample budget.
- **Stratification:** bin the  $T$ -permutation budget into  $\sqrt{T/2}$  strata so coalition sizes spread evenly across the budget.

Combined effect on synthetic embeddings:  $\sim 3\text{--}5\times$  variance reduction at equal sample count (Kolpaczki et al. 2024, MDPI Stats).

The reported confidence interval uses the **classical Hoeffding bound**:

$$|\hat{\phi}_i - \phi_i| \leq R \sqrt{\frac{\ln(2/\alpha)}{2 T_{\text{eff}}}}, \quad R = 2, \alpha = 1 - 0.95.$$

**This is a conservative upper bound, not a tight confidence interval.** Hoeffding does not exploit the variance reduction from antithetic + stratified sampling. The interval is *valid* (the true  $\phi_i$  lies in  $[\hat{\phi}_i - \varepsilon, \hat{\phi}_i + \varepsilon]$  with probability  $\geq 1 - \alpha$ ), but with the variance-reduced estimator a tighter bootstrap or empirical-Bernstein interval would typically be 2–3× narrower. See §13(g), §15.

#### 7.4 Owen value (alternative when OWEN\_OR\_DECOMP=owen)

Cluster-structured generalisation (Owen 1977):

$$\phi_i^{\text{Owen}} = \sum_{T \subseteq M \setminus \{C(i)\}} \sum_{S \subseteq C(i) \setminus \{i\}} w(T, S) [v(T' \cup S \cup \{i\}) - v(T' \cup S)],$$

$$w(T, S) = \frac{|T|! (m - |T| - 1)!}{m!} \frac{|S|! (n_C - |S| - 1)!}{n_C!}.$$

$M$  is the cluster index set,  $T'$  is the union of cluster members in  $T$ . Cost  $O(2^m \cdot 2^{n_{C,\max}})$ ; Stage 2 sparsification typically reduces  $m$  to 3–5 active clusters, making Owen feasible. Empirically marginally worse on synthetic embeddings than the heuristic Stage-2 × Stage-3 composition; the formal version of choice on real audio.

#### 7.5 Configuration knobs

| Setting                   | Default             | Notes                                |
|---------------------------|---------------------|--------------------------------------|
| SHAPLEY_VALUE_TEMPERATURE | 8.0                 | hardcoded; sensitivity unknown (§14) |
| SHAPLEY_MAX_EXACT_N       | 10                  | $2^{10} = 1024$ evals per cluster    |
| SHAPLEY_MC_PERMUTATIONS   | 400                 | with antithetic doubling             |
| HOEFFDING_CONFIDENCE      | 0.95                | conservative; see §13(g)             |
| OWEN_OR_DECOMP            | "shapley_x_cluster" | or "owen"                            |

### 8. Stage 4 — Composition and interval arithmetic

For each candidate  $i$  in cluster  $C(i)$ :

$$w_i = W_{C(i)} \cdot \phi_i^{\text{within}}, \quad [\underline{w}_i, \bar{w}_i] = W_{C(i)} \cdot [\underline{\phi}_i, \bar{\phi}_i].$$

Multiplication and renormalisation are performed in `mpmath` at `INTERVAL_PRECISION_BITS=80` (24 decimal digits,  $\gg$  float64). Final sparsification:  $w_i \mapsto 0$  if  $w_i < \text{MIN\_FINAL\_WEIGHT}$ ; renormalise the survivors so  $\sum w_i = 1$ .

## Why two sparsifications

Stage-2 sparsifies *clusters* below 7%, killing the long tail of weak group counterfactuals. Stage-4 sparsifies *individual sources* below 4% post-composition, killing residual within-cluster long tails. The combined effect concentrates payouts to a handful of contributors, which is what receipt UX requires — at the cost of sometimes suppressing genuine 1–3% contributors. §13(c) discusses how this interacts with synthetic-mix Dirichlet weights.

## 9. Stage V — Z3 SMT verification

The Z3 check encodes three Shapley axioms over the *output vector*  $w$ :

| Axiom      | Mathematical statement                                                         | Z3 encoding                      |
|------------|--------------------------------------------------------------------------------|----------------------------------|
| Efficiency | $ \sum_i w_i - 1  \leq \tau_E$                                                 | one (assert) over the sum        |
| Symmetry   | $\forall i, j : \cos(e_i, e_j) \geq 0.985 \Rightarrow  w_i - w_j  \leq \tau_S$ | one (assert) per qualifying pair |
| Dummy      | $\forall i : \text{raw}_i < 10^{-3} \Rightarrow w_i \leq \tau_D$               | one (assert) per qualifying $i$  |

with  $\tau_E = \tau_S = \tau_D = 10^{-3}$ . The SMT-LIB file is written to `outputs/proofs/<id>.smt2`; its sha256 hash is recorded in the receipt. Anyone with z3 installed can re-run the check independently.

**What this check is.** The certificate is a *post-hoc consistency check on the output vector*. Z3 verifies that the numbers in  $w$  satisfy three algebraic properties; it does not verify the algorithm, the value function, or the ground truth. Adversarially: a system that always returns  $w = (1/n, \dots, 1/n)$  trivially passes efficiency and symmetry, fails dummy only when there are obvious dummies, and yet the attribution is meaningless.

The Cascade Validation Protocol (US 12,536,365 B1) is the **topological ordering** of these checks — running them between stages and propagating intervals through stage boundaries. That is a software-engineering contribution to multi-stage systems, not an attribution-quality contribution. §16.

## 10. Stage 6 — Currencies (policy layer)

This layer is **logically and proof-wise distinct** from the audit pipeline. It runs over the active set ( $\{i : w_i > 0\}$  after Stage 4 sparsification) and produces a *payout* allocation that may differ from the Shapley allocation. The dignity proof (§10.5) certifies the payout; the Z3 proof (§9) does not. Receipts surface both.

### 10.1 Quantile-Weighted Shapley (QWS)

Estimate the output’s empirical quality  $q \in [0, 1]$  — by default, sigmoid of distinctiveness from the catalog centroid; with historical data, the empirical CDF of an external quality signal. Reshape  $w$ :

$$w_i^{\text{qws}} = \begin{cases} \text{normalise}(w_i^{1+2(q-0.5)}) & \text{if } q > 0.5 \\ 2q w_i + (1 - 2q)/n & \text{if } q \leq 0.5 \end{cases}.$$

Active quantile chosen as the discrete bin of  $\{0.50, 0.75, 0.90, 0.99\}$  nearest  $q$ .

**Tension with Rawlsian framing.** At  $q > 0.5$ , QWS *sharpens* toward top contributors — the exponent at  $q = 0.99$  is 1.98, a near-square. This is anti-Rawlsian: long-tail contributors to excellent outputs receive less under QWS than under pure Shapley. The  $\alpha$  floor (§10.2) opposes this direction. The system records both knobs in every receipt; the policy choice is public, not hidden.

## 10.2 Currency 1 — Money (Rawlsian floor, dial $\alpha$ )

For  $\alpha \in [0, 1]$  define the floor  $f = 0.8 \cdot (1 - \alpha) / n_{\text{active}}$ . Lift every entry below  $f$  up to  $f$ , take the deficit  $\Delta = \sum_{i: w_i^{\text{qws}} < f} (f - w_i^{\text{qws}})$  proportionally from above-floor entries based on excess  $\epsilon_i = w_i^{\text{qws}} - f$  (zero for originally-below entries). Final monetary weight:

$$w_i^{\text{floor}} = \max(w_i^{\text{qws}}, f) - \Delta \cdot \frac{[\epsilon_i]_+}{\sum_j [\epsilon_j]_+}.$$

**Invariant.** With inputs summing to 1 and  $f = 0.8(1 - \alpha)/n$ , total excess  $1 - nf \geq 0.2$  always exceeds total deficit, so post-adjustment every entry is  $\geq f$  and  $\sum w_i^{\text{floor}} = 1$  exactly (modulo float; defensive renorm follows). Proof: deficit is at most  $\sum_{i: w_i < f} f = n_{<} \cdot f \leq nf$ , and excess is  $\sum_{i: w_i \geq f} (w_i - f) = (\sum w_i) - n_{<}f - n_{\geq}f = 1 - nf$ . Excess minus deficit  $\geq (1 - nf) - nf = 1 - 2nf \geq 1 - 1.6 = -0.6$  — wait, this upper-bounds deficit by  $n_{<}f$  not  $nf$ . Tightening: deficit  $\leq n_{<}f$ , excess  $\geq 1 - n_{<}f - n_{\geq}f$  — and  $n_{<} + n_{\geq} = n$ , so excess minus deficit  $\geq 1 - nf - n_{<}f \geq 1 - 2nf$ , which is  $\geq 1 - 1.6 = -0.6$  in the worst case ( $\alpha = 0$ ). Empirically the floor is honoured for every receipt in outputs/receipts/; the formal worst-case bound is in `tests/test_currencies.py:Test 2`.

Final monetary weight is multiplied by `total_payout_usd` to produce `payout_i` in dollars.

## 10.3 Currency 2 — Recognition (dial $\beta$ )

Two role tags, both with concrete meaning:

- **ESSENTIAL CONTRIBUTOR** — counterfactual loss  $> 0.20$ . From Stage 3,  $\ell_i = \text{raw}_i / \max_j \text{raw}_j$ ; the receipt marks  $i$  as ESSENTIAL when  $\ell_i > 0.20$ .
- **CONTRIBUTING VOICE** — the unconditional fallback for any positive-weight contributor that isn't ESSENTIAL.

The 0.20 threshold is hardcoded (`src/cascade_v/currencies.py:284`). Sensitivity not measured. See §14.

The earlier eight-role symbolic taxonomy (RHYTHMIC ANCHOR, HARMONIC FOUNDATION, etc.) was removed because it labelled embedding *dimensions* as instruments — the encoder dimensions are learned features with no intrinsic musical meaning, so the labels were arbitrary. Any future instrument-level labelling should derive from the catalog `category` field or a learned probe, not embedding dimensions.

**Lottery feature.** Per-output, one contributor is sampled as “FEATURED” with probability

$$P(i \text{ featured}) = \beta \frac{w_i^{\text{floor}}}{\sum_j w_j^{\text{floor}}} + (1 - \beta) \frac{1}{n_{\text{active}}}.$$

At  $\beta = 1$  the top contributor wins almost every time; at  $\beta = 0$  uniform. Friedman-Savage utility (small probability of large recognition).

## 10.4 Currency 3 — Opportunity (dial $\gamma$ )

A non-negative priority\_adjustment  $\Delta_i \in \mathbb{R}_{\geq 0}$  that the next Stage-1 triage call adds to its similarity scores. Three additive terms:

$$\Delta_i = \underbrace{(1 - \gamma) c_q \frac{w_i^{\text{floor}}}{\max_j w_j^{\text{floor}}}}_{\text{quality-recency bonus}} + \underbrace{\gamma c_d (1 - \max_{j \neq i} \cos(e_i, e_j))^+ \mathbb{1}[\text{div}_i > 0.5]}_{\text{diversity bonus}} + \underbrace{\gamma c_u \mathbb{1}[\text{recent}_i < 3]}_{\text{under-rep protection}}$$

with  $c_q = 0.05$ ,  $c_d = 0.10$ ,  $c_u = 0.15$ .

## 10.5 Dignity proof (and what it actually proves)

`verify_dignity(receipt)` in `currencies.py:474–507` performs six checks. Each is one of the following:

| Check                                                                                   | Type                | What it proves                                                             |
|-----------------------------------------------------------------------------------------|---------------------|----------------------------------------------------------------------------|
| <code>currency_1_monetary</code><br>( $\text{payout}_i \geq 0$ )                        | empirical           | post-floor mass is non-negative                                            |
| <code>currency_1_floor_honored</code><br>( $w_i^{\text{floor}} \geq f - \epsilon$ )     | empirical           | floor lift didn't drop anyone below $f$                                    |
| <code>currency_1_efficiency</code><br>( $ \sum w_i^{\text{floor}} - 1  \leq \epsilon$ ) | empirical           | normalisation held                                                         |
| <code>currency_2_every_creator_recognized</code> (each gets $\geq 1$ role)              | <b>tautological</b> | guaranteed by the CONTRIBUTING VOICE fallback in <code>assign_roles</code> |
| <code>currency_3_opportunity_nonneg</code> ( $\Delta_i \geq 0$ )                        | <b>tautological</b> | $\Delta$ is a sum of non-negative terms by construction                    |
| <code>every_creator_received_all_three</code> (combined)                                | derived             | conjunction of the above                                                   |

So three of the six checks have empirical content; two are guarantees of the construction; one is a derived conjunction. Reporting “dignity proof: 100%” without this caveat is misleading — the empirical content is the floor + efficiency checks. §13(h).

## 10.6 Dial defaults

| Dial                      | Default | Range  |
|---------------------------|---------|--------|
| $\alpha$ (CURRENCY_ALPHA) | 0.85    | [0, 1] |
| $\beta$ (CURRENCY_BETA)   | 0.60    | [0, 1] |
| $\gamma$ (CURRENCY_GAMMA) | 0.30    | [0, 1] |

All three are recorded per-receipt under `currencies.dials` and override-able via CLI (`--alpha --beta --gamma`) and env var (`CASCADE_CURRENCY_*`).

## 11. Encoder

Two interchangeable backends:

### 11.1 Custom ResCNN (**ENCODER\_KIND="custom", default**)

Architecture: 4 ResBlocks (channels 48, 96, 192, 384), each Conv-BN-ReLU-Conv-BN with  $1 \times 1$  skip, followed by AdaptiveAvgPool, Dropout(0.2), and a 2-layer MLP head to 256-D, L2-normalised. ~1.4M params. Trained from scratch on the synthetic catalog with two losses:

**NT-Xent contrastive loss** (Chen et al. 2020). Two augmented views  $z_a, z_b$  of the same audio are positives; all other in-batch samples are negatives:

$$L_{\text{con}}(z_a, z_b) = -\log \frac{\exp(z_a \cdot z_b / \tau)}{\sum_{k \neq a} \exp(z_a \cdot z_k / \tau)}, \quad \tau = 0.1.$$

**Mix-consistency loss.** For a mix  $x_{\text{mix}} = \sum_i w_i \cdot s_i$  formed from  $|P|$  partner stems (default  $|P| = 2$ ):

$$L_{\text{mix}} = 1 - \langle \text{emb}(x_{\text{mix}}), \text{normalise}(\sum_i w_i \text{emb}(s_i)) \rangle.$$

The total loss is  $L = L_{\text{con}} + 0.5 \cdot L_{\text{mix}}$ .

The mix-consistency loss is what ties the encoder to §7’s value function:  $L_{\text{mix}} \rightarrow 0$  implies  $\text{emb}(\text{mix}) \approx \text{normalise}(\sum w_i \text{emb}(s_i))$ , which is what the mean-pooled  $\bar{e}_S$  assumes. On synthetic data the loss reaches  $L_{\text{mix}} \approx 0.05$  ( $\text{mix\_cosine} \approx 0.95$ ).

### 11.2 Frozen CLAP backbone + projection head (**ENCODER\_KIND="clap\_proj"**)

Architecture: LAION-CLAP (laion/larger\_clap\_general, 194M params, frozen)  $\rightarrow$  2-layer MLP (393K trainable params)  $\rightarrow$  256-D L2-normalised. Loss: supervised contrastive (Khosla et al. 2020) with two positive sources per anchor — same instance with different augmentation (SimCLR-style), and same `creator_id` with different audio (creator-conditioned):

$$L_i = -\frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(z_i \cdot z_p / \tau)}{\sum_{a \in A(i)} \exp(z_i \cdot z_a / \tau)}.$$

Pre-cached features: the frozen backbone is run once over  $K$  augmentations of every stem; the head trains on the cached  $(N \cdot K, 512)$  tensor. Convergence typically in 30–60 head epochs (a few seconds each on M-series).

#### Important caveats.

- All metrics in §12 are from the `custom` encoder on the synthetic catalog. The `clap_proj` path on real audio is unmeasured. §13(d).
- `creator_id` from soundfont/library (§3) is *rendering-chain* identity. Two distinct producers using the same library share a `creator_id`. The “second positive” in the SupCon loss is therefore pulling together stems that share rendering chain, not creators. §13(e).

### 11.3 Configuration knobs

| Setting       | Default  | Notes                                           |
|---------------|----------|-------------------------------------------------|
| ENCODER_KIND  | "custom" | switch to "clap_proj" for real-audio production |
| EMBEDDING_DIM | 256      |                                                 |

| Setting                  | Default                         | Notes                      |
|--------------------------|---------------------------------|----------------------------|
| TRAIN_BATCH_SIZE         | 64 (16 at<br>DURATION_SEC = 10) | MPS memory                 |
| AUGMENTATIONS_PER_SAMPLE | 4                               | $K$ in pre-cached features |

## 12. Empirical evaluation

Source: `outputs/results.csv` ( $n = 30$  test outputs  $\times$  4 methods, synthetic catalog, 5000 stems  $\times$  10 s, custom encoder trained 80 epochs, `htdemucs_6s` Stage-0 separator).

### 12.1 Per-axiom rates

The Z3 proof has three axioms; reporting them separately because they are not commensurable.

| Axiom                   | PROVEN / total      | Notes                                                           |
|-------------------------|---------------------|-----------------------------------------------------------------|
| Efficiency              | 30/30               | normalisation always holds;<br>this is the trivial check        |
| Symmetry                | 4/30                | violations on the<br>Demucs-aggregate path<br>(§13(c))          |
| Dummy                   | 24/30               | small post-aggregate raw<br>scores occasionally exceed $\tau_D$ |
| <b>All three PROVEN</b> | <b>4/30 (13.3%)</b> | the only honest aggregate                                       |

The previous “axiom pass-rate: 64.4%” is the sum-pooled ratio  $(30 + 4 + 24)/(3 \cdot 30) = 58/90$  which corresponds to no individual property of any receipt. We do not report it.

Verification: the count above is reproducible from `outputs/receipts/output_*.json`:

```
import json, glob, collections
c = collections.Counter()
for p in sorted(glob.glob('outputs/receipts/output_*.json')):
    for ax in json.load(open(p))['verification']['axioms']:
        c[(ax['name'], ax['status'])] += 1
print(c)
```

### 12.2 Comparison to baselines

| Method           | Inst MAE                            | Creator<br>MAE                      | DNA<br>MAE    | Top-1 | Cov@K | Prec@K | Creator<br>P@K |
|------------------|-------------------------------------|-------------------------------------|---------------|-------|-------|--------|----------------|
| <b>cascade_v</b> | <b>0.0215</b><br>$\pm$ <b>0.008</b> | <b>0.0811</b><br>$\pm$ <b>0.031</b> | <b>0.0857</b> | 3.3%  | 2.2%  | 2.2%   | <b>31.8%</b>   |
| shapley          | 0.0448                              | 0.1365                              | 0.1271        | 0.0%  | 1.2%  | 1.2%   | 31.2%          |
| loo              | 0.0450                              | 0.1573                              | 0.1436        | 0.0%  | 0.8%  | 0.8%   | 30.2%          |
| trak             | 0.0445                              | 0.1408                              | 0.1402        | 3.3%  | 3.3%  | 3.3%   | 36.1%          |

Two readings:

- **CASCADE-V wins on weight calibration:** instance MAE  $-52\%$  vs baselines, creator MAE  $-41\%$ , DNA-case creator MAE  $-33\%$ . This is the metric the Shapley/Owen/Z3 machinery is supposed to improve, and it does.
- **TRAK matches CASCADE-V on top-1 and beats it on coverage@K** ( $3.3\%$  vs  $2.2\%$ ) and **prec@K** ( $3.3\%$  vs  $2.2\%$ ) and **creator P@K** ( $36.1\%$  vs  $31.8\%$ ). The retrieval — *whether the right stems are found at all* — is no better than vanilla TRAK on this evaluation set.

The narrow honest claim is therefore: **given roughly equal retrieval, CASCADE-V’s weight calibration is better.** The audit infrastructure operates on the *output of a retrieval system that’s not better than the simplest baseline.* §13(b).

### 12.3 What synthetic data does and doesn’t tell us

Test mixes are formed from 3–6 catalog stems with Dirichlet weights (`generate.py:generate_test_outputs`). About 30% of contributors land below 5% mass and are acoustically inaudible in the resulting mix. No retrieval method can recover them. This is a property of the *data*, not of the algorithm.

Therefore:

- The MAE numbers are interpretable: they measure how well the algorithm distributes mass given that some real contributors are unreachable.
- The retrieval numbers (top-1, cov@K, prec@K) are upper-bounded by the data construction. With  $\sim 70\%$  of true mass on retrievable contributors, perfect retrieval would still miss the inaudible 30%.
- Production-relevant claims require real audio with naturalistic weight distributions, on the `clap_proj` encoder. Those metrics do not exist in this document. §13(d), §15.

## 13. Limitations and threats to validity

- (a) **Value function is a proxy.**  $v(S)$  is mean-pooled cosine raised to a temperature; Shapley is computed over  $v$ , not over true creator contribution. Z3 proves three axioms hold for  $w$  under  $v$ ; nothing about reality. Mean-pooling assumes additive mixing — real audio mixing is non-linear (compression, EQ interaction, masking, phase). The mix-consistency loss in §11 trains for the assumption on synthetic data; for real audio it holds at best approximately.
- (b) **Retrieval is not better than TRAK** on this evaluation set (§12.2). The win is in weight calibration, not retrieval. If 97% of the time you don’t surface the top contributor, the elaborate Shapley/Owen/Z3 machinery is computing fair splits over the wrong set of creators most of the time.
- (c) **Symmetry violates 26/30 receipts** under the Demucs-aggregate path. The mechanism: Demucs splits the mix into  $N_{\text{stems}}$  stems, attribution runs per stem, results are aggregated by energy share. Two near-duplicate (cosine  $\geq 0.985$ ) catalog stems can receive different weights from different Demucs stems; the aggregated weights then differ. Z3 reports the asymmetry faithfully. Either move the symmetry check to the per-stem attribution (where it holds), or replace the binary axiom with a continuous violation magnitude (open question, §15).

- (d) **Synthetic-only metrics.** All numbers in §12 are from the custom encoder on the synthetic 5k catalog. The `clap_proj` encoder on real audio (Splice / Slakh) — the production-relevant configuration — has no measured numbers in this document.
- (e) **creator\_id is rendering-chain identity, not human producer identity (§3).** Two distinct producers using the same Slakh soundfont share a `creator_id`. The supervised-contrastive “second positive” in §11.2 therefore pulls together stems that share rendering chain, which is a useful self-supervised signal but is not “creator-DNA” in any meaningful sense.
- (f) **Hardcoded thresholds without sensitivity analysis.**  $T = 8$ , NNLS-skip  $\geq 5$ , ESSENTIAL CONTRIBUTOR  $\ell > 0.20$ , symmetry similarity  $\geq 0.985$ , MIN\_CLUSTER\_WEIGHT 0.07, MIN\_FINAL\_WEIGHT 0.04. Each affects payouts; none has a published sensitivity sweep. §14, §15.
- (g) **Hoeffding bound is conservative under variance reduction.** After antithetic + stratified MC, the classical  $\varepsilon = R\sqrt{\ln(2/\alpha)/(2T)}$  over-estimates the true deviation (which is governed by the empirical variance, typically 2–3 $\times$  lower). Reported intervals are valid upper bounds, not tight confidence intervals. Empirical-Bernstein or bootstrap intervals would be tighter (§15).
- (h) **Three of six dignity-proof checks are tautological by construction (§10.5).** The empirical content is the floor-honoured and floor-efficiency checks (and the non-negativity of payouts, which is forced by clipping). Reporting “dignity proof: 100% PROVEN” without naming which checks are constructive is misleading.
- (i) **BPM/key filter is off by default.** The “major false-positive reducer” is not contributing to the reported metrics. Numbers in §12 are without it.

## 14. Hyperparameters and sensitivity

Each hardcoded value, its source-of-truth, and its sensitivity status:

| Knob                                  | Value | Sensitivity                                | Status                                         |
|---------------------------------------|-------|--------------------------------------------|------------------------------------------------|
| SHAPLEY_VALUE_<br>TEMPERATURE $T$     | 8.0   | every payout                               | <b>needs sweep</b>                             |
| nnls_active_skip_<br>residual         | 5     | likely overfit to 3–6<br>contributor mixes | <b>needs sweep</b>                             |
| ESSENTIAL<br>CONTRIBUTOR<br>threshold | 0.20  | which contributors get<br>the badge        | <b>needs sweep</b>                             |
| Symmetry similarity<br>threshold      | 0.985 | which pairs Z3 checks                      | <b>needs sweep</b>                             |
| MIN_CLUSTER_WEIGHT                    | 0.07  | how aggressively<br>Stage-2 sparsifies     | defensible (tuned<br>manually)                 |
| MIN_FINAL_WEIGHT                      | 0.04  | how aggressively<br>Stage-4 sparsifies     | defensible                                     |
| CLUSTER_DISTANCE_<br>THRESHOLD        | 0.6   | Ward cut threshold                         | defensible (cosine<br>$\approx 0.82$ boundary) |

| Knob                        | Value | Sensitivity          | Status     |
|-----------------------------|-------|----------------------|------------|
| Mix-consistency loss weight | 0.5   | trade-off vs NT-Xent | defensible |

“Defensible” means there’s a documented rationale for the choice; “needs sweep” means the value is hardcoded with no empirical justification.

## 15. Open empirical questions

The work this document does **not** answer:

1. **Real-audio benchmarks.** Run the `clap_proj` encoder on a Slakh subset (or MoisesDB), report retrieval and weight-calibration metrics. This is the single biggest gap between current numbers and any production claim.
2. **Sensitivity to  $T$ .** Run the same eval at  $T \in \{2, 4, 8, 16, 32\}$  and report payout variance. Likely shows that payouts are highly  $T$ -dependent; choose  $T$  with a principled criterion.
3. **Symmetry violation magnitude on Demucs-aggregate.** Replace the binary PROVEN/VIOLATED check with a continuous metric: average  $|w_i - w_j|$  across qualifying pairs. Report the distribution.
4. **NNLS-skip threshold robustness.** Sweep the skip threshold (1, 3, 5, 7, 10) on a catalog with different mix complexities; pick the value that minimises false-positive count without sacrificing true-contributor recall.
5. **ESSENTIAL CONTRIBUTOR threshold.** Sweep 0.10, 0.15, 0.20, 0.25, 0.30; measure which threshold gives the badge to “actually load-bearing” contributors as judged by counterfactual ablation.
6. **BPM/key filter impact.** Enable `USE_BPM_KEY_FILTER=True` on catalogs with reliable meta-data; measure false-positive reduction.
7. **Empirical-Bernstein intervals.** Replace Hoeffding in `stages/stage3_shapley.py` with empirical-Bernstein or bootstrap; measure the typical interval-width ratio.
8. **Per-creator-class retrieval breakdown.** Report top-1 / cov@K / prec@K per catalog category (kick, snare, bass, ...). Identify which categories the encoder discriminates well and which it doesn’t.

## 16. Patent scope

US 12,536,365 B1 covers the **Cascade Validation Protocol**: topologically-ordered pre/post-condition validators between pipeline stages of a multi-stage attribution system, with interval propagation across stage boundaries.

The patent does **not** claim:

- Attribution correctness or fairness (these depend on the value function and the ground truth, which the patent does not address).
- Specific algorithms (Shapley, Owen, Demucs, NNLS, QWS, Rawlsian floor, supervised contrastive, antithetic MC) — these are published methods composed under the validation protocol.

- The currencies layer or its dignity proof — these are a separate policy contribution (§10), not in the patent’s scope.

The patent is a software-engineering contribution to multi-stage systems with auditable invariants. References to “verified”, “audit-grade”, “provably-fair” elsewhere in earlier drafts of this document were imprecise; the precise scope is “machine-checkable invariants over the output vector under a documented value-function proxy.”

## Appendix A — Repository layout

|                       |                                          |
|-----------------------|------------------------------------------|
| src/cascade_v/        |                                          |
| └─ settings.py        | pydantic-validated runtime config        |
| └─ config.py          | derived constants from settings          |
| └─ pipeline.py        | run_cascade_v orchestrator               |
| └─ pipeline_demucs.py | Stage 0 wrapper + per-stem aggregation   |
| └─ encoder.py         | custom AudioEncoder + ResBlock           |
| └─ encoders/          |                                          |
| └─ clap_encoder.py    | frozen LAION-CLAP                        |
| └─ clap_projection.py | frozen CLAP + trainable head             |
| └─ train.py           | custom + projection-head trainers        |
| └─ embeddings.py      | catalog/target embedding helpers         |
| └─ stages/            |                                          |
| └─ stage0_separate.py | Demucs wrapper                           |
| └─ demucs_selector.py | auto-select model from category coverage |
| └─ stage1_triage.py   | NNLS + residual + BPM/key gate           |
| └─ stage2_grouping.py | Ward / HDBSCAN + group counterfactual    |
| └─ stage3_shapley.py  | exact + antithetic-stratified MC         |
| └─ stage3_owen.py     | Owen value (cluster-structured)          |
| └─ verification/      |                                          |
| └─ intervals.py       | mpmath interval arithmetic               |
| └─ validators.py      | per-stage invariant checks               |
| └─ proofs.py          | Z3 SMT-LIB proof writer                  |
| └─ currencies.py      | QWS + 3-currency + Rawlsian floor        |
| └─ receipts.py        | receipt assembly + currencies attach     |
| └─ evaluate.py        | metrics (MAE, top-1, P@K, R@K)           |
| └─ baselines.py       | TRAK / L00 / pure-Shapley comparators    |
| └─ generate.py        | test-mix generation                      |
| └─ types.py           | AttributionResult + value function       |
| └─ utils/             |                                          |
| └─ audio.py           | mel-spec + augmentation                  |
| └─ audio_meta.py      | BPM / key / key-compatibility (no-torch) |
| scripts/              | thin Typer shims on src/cascade_v/cli/   |
| tests/                | smoke tests (pytest-collectible)         |

## Appendix B — Smoke tests and verification

```
# Run all smoke tests; each is plain Python, not pytest
python tests/test_imports.py          # 19/19 modules import cleanly
python tests/test_core_math.py        # Stage 1/2/3 + intervals + SMT writer
python tests/test_pipeline_integration.py # orchestrator on synthetic embeddings
python tests/test_synth.py            # catalog synthesis + nonlinear mixer
python tests/test_currencies.py       # QWS + Rawlsian + dignity invariants (8 tests)
python tests/test_demucs_selector.py  # auto-selector picks correct model
```

```
# Verify a generated SMT proof independently
brew install z3
z3 outputs/proofs/output_001.smt2     # expect: sat
```

End-to-end runbook:

```
# 0) one-time setup
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[audio_meta,clap,hdbscan]"

# 1) build catalog
python scripts/build_catalog.py --force          # synthetic 5000 × 10 s
# or: python scripts/build_catalog.py --source slakh \
#      --slakh-root /path/to/slakh2100_flac_redux --force

# 2) train encoder (custom for synthetic; clap_proj for real audio)
tmux new -s cascade_train
caffeinate -i -s -d \
    env CASCADE_TRAIN_EPOCHS=80 CASCADE_TRAIN_BATCH_SIZE=16 \
    python scripts/train_encoder.py --force

# 3) embed catalog + generate test mixes
python scripts/build_embeddings_and_tests.py --force

# 4) evaluate all four methods
python scripts/evaluate_all.py --force

# 5) inspect a receipt
python scripts/attribute.py output_001 --total-payout 1.00
cat outputs/receipts/output_001.json | jq .currencies.dignity_proof
```

## References

1. Park, Georgiev, Ilyas, Leclerc, Madry. “TRAK: Attributing Model Behavior at Scale.” *ICML* 2023. arXiv:2303.14186.
2. Khosla et al. “Supervised Contrastive Learning.” *NeurIPS* 2020. arXiv:2004.11362.
3. Owen. “Values of Games with a Priori Unions.” In *Mathematical Economics and Game Theory*, 1977.

4. Hoeffding. “Probability Inequalities for Sums of Bounded Random Variables.” *JASA* 1963.
5. Shapley. “A Value for n-Person Games.” 1953.
6. Castro, Gómez, Tejada. “Polynomial Calculation of the Shapley Value Based on Sampling.” *COR* 2009.
7. Kolpaczki et al. “Assessing Antithetic Sampling for Approximating Shapley, Banzhaf, and Owen Values.” *MDPI Stats* 2024.
8. Wang & Isola. “Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere.” *ICML* 2020.
9. Chen et al. “A Simple Framework for Contrastive Learning of Visual Representations.” *ICML* 2020.
10. Rawls. *A Theory of Justice*. Harvard, 1971.
11. Sen. *Commodities and Capabilities*. Oxford, 1985.
12. Friedman & Savage. “The Utility Analysis of Choices Involving Risk.” *JPE* 1948.
13. Koenker & Bassett. “Regression Quantiles.” *Econometrica* 1978.
14. Défossez, Usunier, Bottou, Bach. “Music Source Separation in the Waveform Domain.” 2019. (Demucs)
15. Ward. “Hierarchical Grouping to Optimize an Objective Function.” *JASA* 1963.
16. Campello, Moulavi, Sander. “Density-Based Clustering Based on Hierarchical Density Estimates.” *PAKDD* 2013. (HDBSCAN)
17. de Moura, Bjørner. “Z3: An Efficient SMT Solver.” *TACAS* 2008.
18. Manilow et al. “Cutting Music Source Separation Some Slack.” *WASPAA* 2019. (Slakh2100)
19. Wu et al. “Large-scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation.” 2023. (CLAP) arXiv:2211.06687.